⚖️ Find The Balance: Prescriptive vs. Generic

🛑 1. Avoid: Overly Prescriptive (Counterproductive) Rubrics

Ask yourself: If a model provides a different but technically sound solution (e.g., a different design pattern or library), would this rubric force a failure?

- **Why it's bad:** It creates "noisy" data by penalizing industry-standard answers simply because they didn't use your one preferred method.
- **The Only Exceptions:** If the prompt **explicitly** demands a specific method (e.g., "Use a `for` loop") or if there is only **one** secure way to handle the task.

☁️ 2. Avoid: Overly Vague (Generic) Rubrics

Ask yourself: If you swapped this Python prompt for a Java or SQL prompt, would the rubric still work perfectly?

- **Why it's bad:** "Filler" rubrics like "code is readable" or "provides a summary" tell us nothing about the actual technical logic (e.g., Python string methods).

💡 2 Simple "Technical Teeth" Tricks: wording really matters here!

To write a high-quality rubric, you must provide the **ground truth** without being a dictator. Use these two strategies to get it right:

🛠️ Strategy 1: Always Use the word "LIKE/e.g." in your rubrics!

Instead of forcing one method, provide a comprehensive list of valid approaches. This gives the LLM Judge a "Standard of Truth" while remaining flexible.

- ❌ **Overly Prescriptive:** "The response uses **initializer injection** to demonstrate Dependency Injection."
- ❌ **Overly Vague:** "The response uses **a valid method** to demonstrate Dependency Injection."
- ✅ **Perfect Balance:** "The response demonstrates at least one standard form of Dependency Injection (**e.g., Initializer, Property, or Method injection**)."

📚 Strategy 2: Give Comprehensive Examples

When listing methods, don't just look at what the 4 model responses did. Use your own knowledge and consider other valid, industry-standard ways to solve the problem. Give multiple concrete examples and options

- ❌ **Reiterating the Prompt:** "The response explains how to decode a `.rpgsave` file."
- ✅ **Adding Technical Teeth:** "The response explains how to decode a `.rpgsave` file (**e.g., reverse-engineer the format, then deserialize it with the game engine's save schema**)."

📝 Concrete Examples (Based on QC Fails)

Feature	❌ Bad (Prescriptive or Vague)	✅ Better (Balanced with "LIKE/e.g.")
Formatting	"The response uses bullet points and checkmarks."	"The response uses a clear, structured list **(e.g., bullet points, numbered lists, or checkmarks)."**
Logic/Architecture	"The code must use two distinct `DbContext` classes."	"The response separates read/write concerns **(e.g., via two `DbContexts`, a Context Factory, or a Repository pattern)."**
Code Specifics	"The response uses valid DOM property names."	"The response fixes HTML-to-React attribute errors **(e.g., changing `class` to `className` or `for` to `htmlFor`)."**

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.862Z

Security/Libs	"The code must use the cryptography library."	"The response utilizes a secure, modern library **like** cryptography (avoiding outdated ones **like** PyCrypto)."
Closing Tone	"The response MUST NOT contain any text outside the 3 technical topics."	"The response focuses on the 3 technical topics, allowing for standard conversational fillers **(e.g., 'Let me know if you need more help')."**

🛑 Error Type: Miscategorization of Rubric Criteria

Definition: Using a designation (mandatory vs nice-to-have) that is not appropriate for the type of criterion.

Remember:

- **Code** rubrics may be mandatory OR nice-to-have.
- **IF** rubrics must be **mandatory.**
- **QoL** rubrics must be **nice-to-have.**

🛑 Error Type: Counterproductive Rubrics (Overly Prescriptive)

Definition: Creating criteria that are too narrow, rigid, or opinionated, forcing the model to follow one specific method when multiple valid technical solutions exist. This leads to "False Fails" where a model provides a correct, industry-standard answer but is penalized for not guessing the attempter's preferred sub-method.

The "Prescriptive" Test: Ask yourself: "If the model provides a different but technically sound solution (e.g., a different design pattern or library), would this rubric force me to fail it?" If yes, the rubric is likely counterproductive and lacks the necessary flexibility for high-quality AI training.

Contextual Exceptions (When it's a Non-Fail):

- **Explicit Constraints:** If the user **specifically asks** for a certain method in the prompt (e.g., "Use a `for` loop, not a `while` loop"), then a restrictive rubric is mandatory.
- **Security/Safety:** If one specific method is the *only* secure way to handle the task (e.g., using parameterized queries to avoid SQL injection), a prescriptive rubric is required.

Example Case: Prompt: “I’m an iOS developer and an interviewer asked me: Do you know what Dependency Injection is?”

❌ **Bad Practice (Counterproductive/Rigid): Criterion 1:** "The response includes a Swift code example demonstrating **initializer injection** to show concrete instantiation for production."

- **Why it's bad:** Initializer injection is just one flavor of DI. If a model provides an excellent explanation using **Property Injection** or **Method Injection**, it would fail this rubric despite being 100% technically correct. This creates "noisy" data for model training.

✅ **Good Practice (Flexible yet Technical): Criterion 1:** "The response provides a Swift code example demonstrating **at least one** standard form of Dependency Injection (e.g., Initializer, Property, or Method injection)”

- **Why it's good:** It maintains "technical teeth" by requiring code and the concept of decoupling, but it does not penalize the model for choosing a different (but valid) implementation style.

🛑 Error Type: Generic Rubrics (Boilerplate Criteria)

Definition: Using "copy-paste" criteria that are not tailored to the specific technical task. These are abstract placeholders (like asking for a summary, next steps, or specific formatting) that could apply to almost any prompt regardless of the programming language or problem.

The "Specific to Task" Test: Ask yourself: "If I swapped this Python prompt for a Java or SQL prompt, would this rubric still work perfectly?" If the answer is yes, the rubric is likely too generic and lacks "technical teeth."

Contextual Exceptions (When it's a Non-Fail):

- **Quality of Life (QoL) Threshold:** Generic criteria are acceptable **only** if they are classified as "Quality of Life" and weighted as **"Nice-to-Have."**

Example Case:

- **Prompt:** "How do I split a string in Python using a delimiter?"

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.862Z

- ❌ **Bad Practice (Generic/Boilerplate):**
  - **Criterion 8:** "The response offers follow-up questions or next steps."
  - **Criterion 11:** "The response provides a brief summary or recap."
  - **Criterion 12:** "The response avoids decorative or excessive formatting."
  - **Why it's weak:** These tell us nothing about Python string methods (`.split()`) or delimiters. They are "filler" rubrics.
- ✅ **Good Practice (Task-Specific):**
  - **Criterion 1:** "The response demonstrates the `.split()` method and explains how to pass a specific string delimiter as an argument."
  - **Criterion 2:** "The code correctly handles edge cases, such as splitting a string that does not contain the specified delimiter."
  - **Criterion 3 (Specific QoL):** "The response suggests using `re.split()` if the user needs to split by multiple different characters at once." (This is a "next step" that is actually **specific** to the task).

🛑 Error Type: Incorrect Ratability Assessment (Missing Context)

Definition: Evaluating a task where the prompt is "Unratable" because it lacks conversation history or referred-to files. **The Rule:** If you have to "guess" what the user is talking about because the history is missing, **stop.** Mark it as **Non-Ratable**.

- **Example Case:** User asks: *"App builder without webcontainer instead I am planning to use a template to build a code and deploy it using flyio."*
- ❌ **Bad Practice (Failing):** Proceeding to write rubrics based on a guess of what the "template" or "app builder" is.
- ✅ **Good Practice (Improved):** Mark as **Non-Ratable**. Justification: "Missing Context. The user refers to 'it' and 'a template' without providing the preceding turns or files. We cannot objectively judge code correctness based on a guess."

🛑 Error Type: Bad Rubrics: Prompt Reiteration

Definition: A bad rubric criterion that simply "echoes" or "repeats" what the user already asked for without defining what a successful output looks like.

The Value-Add Test: Ask yourself: "Does this rubric tell the grader how to measure the answer, or just what the question was?" A rubric must add technical teeth to the prompt's request.

Example:

- **Prompt:** "How to decode a .rpgsave file?"
- ❌ **Bad Criterion:** "The response explains how to decode a .rpgsave file."
- ✅ **Good Example (Improved):** "The response identifies the specific compression algorithm (LZ-String) and method (decompressFromBase64) required to deserialize the standard RPG Maker MV save format."

🛑 Error Type: Rubric Non-Specificity

Definition: Using vague, abstract, or overly broad placeholders that could apply to almost any task, making them impossible to evaluate objectively.

Avoid "Boilerplate" Language: Criteria like "code is readable," "uses best practices," or "good formatting" are too subjective for a high-quality audit.

The Component-Based approach: Instead of one broad "correctness" check, break the logic into specific technical components (e.g., specific libraries, error handling for null inputs, or $O(n)$ complexity).

Contextual Exceptions:

- **When "General" is OK:** If a specific point (like explaining a "deep copy") wasn't asked for in the prompt but is included by the model as a helpful addition, a more general QoL rubric is acceptable.
- **Lenience for QoL:** You can be more lenient with Quality-of-Life (QoL) rubrics when the prompt is intentionally open-ended (e.g., "Create an artistic website").
  Example:
- ❌ **Bad Example (Failing):** "The model uses the correct library for the task."
- ✅ **Good Example (Improved):** "The response utilizes the cryptography library (specifically avoiding the deprecated PyCrypto) to ensure AES-256 encryption."

See more examples in the appendix.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.862Z

🛑 Error Type: Code Rewrite Requirements (Golden Response)

Definition: Submitting a "fixed" response (Rewrite) that still fails one or more of the rubric criteria created for the task.

The "Final Pass" Rule: Every single rubric item—whether Mandatory or Nice-to-Have—must be passed by your Golden Response Rewrite.

Example:

- ❌ **Bad Example (Failing):**
  - **Criterion:** "The response must end with an actionable next step or question."
  - **Rewrite:** "...and that is how you implement the API. In summary, use the fetch command." (Fail: No question or next step).
- ✅ **Good Example (Improved):**
  - **Rewrite:** "...and that is how you implement the API. Would you like me to help you write the error-handling block for this `fetch` call next?"

🛑 Error Type: Scoring Guideline Misalignment

Definition: Assigning an overall score (1-5) for the model responses that contradicts the Scoring Rule.

Fail 1 Code/Instruction Rubric: Max Score is 3.

Fail >=2 Code/Instruction Rubrics: Max Score is 2.
The Major Issue: Giving a 4 or 5 to a response that has a syntax error, missing import, or failed technical requirement.
Example:

- ❌ **Bad Example (Failing): Issue:** Response A is missing a critical library import and uses undefined variables.
  - **Assigned Score:** 4 (Incorrect; a response with broken code cannot be a 4).
- ✅ **Good Example (Improved):**
  - **Assigned Score:** 2 (Correct; the response failed 2 technical "Code Rubric" items).

🛑 Error Type: Rubric Redundancy

Definition: Creating multiple criteria that test the same underlying error.

The MECE Principle: Criteria must be **M**utually **E**xclusive and **C**ollectively **E**xhaustive. One error = one failure.

- **Scenario 1 (The Import Trap):** Avoid having one rubric for "proper imports" and another for "no unimported modules."
- **Contextual Exceptions:**
  - **Distinct Logic:** It is okay to have separate rubrics if the errors are technically different (e.g., "Logic Error in Loop" vs. "Syntax Error in Variable Declaration").
- ❌ **Bad Example (Failing):**
  - **Criterion 1:** "All modules are properly imported."
  - **Criterion 2:** "The code contains no references to unimported modules."
- ✅ **Good Example (Improved):**
  - **Criterion 1:** "All required modules (e.g., `pandas`, `numpy`) are imported and correctly referenced in the code."

🛑 Error Type: Missing Coverage (Instruction Gap)

We want in total ~10 rubrics

Definition: Failing to create rubric items for explicit constraints or requirements mentioned in the user's prompt.

Prompt-First Design: The rubric must be a mirror of the prompt. If the prompt asks for "Python," "O(n) complexity," and "a docstring," there must be a rubric for each.

Quick Checklist:

- **Step 1:** Highlight every "Instruction" in the prompt (look for verbs like *use*, *write*, *calculate*, *format*).
- **Step 2:** Ensure there is at least one **Mandatory** rubric item for every highlight.
- **Step 3:** If the prompt asks for a specific library (e.g., `pandas`), the rubric must explicitly name that library, not just say "the correct library."

❌ **Bad Example (Failing):**

The rubric lacks a criterion checking the code actually uses a symbol name to find the function. A script finding a function by hardcoded address could pass the existing criteria.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.862Z

Appendix

Error Category	Issue Description	Bad Example (Vague/Dependent)	Good Example (Specific/Self-Contained)
Generic Rubrics	Using "copy-paste" criteria that are not tailored to the specific technical task. These are abstract placeholders (like asking for a summary, next steps, or specific formatting) that could apply to almost any prompt regardless of the programming language or problem.	"The response offers follow-up questions or next steps." "The response provides a brief summary or recap."	"The response demonstrates the .split() method and explains how to pass a specific string delimiter as an argument." "The response suggests using re.split() if the user needs to split by multiple different characters at once." (**This is a "next step" that is actually specific to the task**).
Golden Response Rewrite	Submitting a "fixed" response (Rewrite) that still fails one or more of the rubric criteria created for the task. The golden response should be a rewrite based on the highest score model response from previous step that you selected.	The golden response is still not perfect and fail some of the criteria.	The final golden response you rewritten should be a PERFECT answer that pass all criteria.
Vague Adjectives	Using subjective words like "appropriate," "correctly," or "valid" which an LLM judge cannot define.	"The response suggests a tech stack that is appropriate for the scale."	"The response suggests a tech stack **such as SQLite and a single Python script** for a scale of <10k records."
Non-Self-Contained	Requiring the judge to refer back to the prompt or external docs to understand the criteria.	"The model follows the specific instructions mentioned in the prompt."	"The model implements the AES-256 encryption using the **cryptography library as requested.**"
Lack of Ground Truth	Failing to specify the actual technical method required, forcing the judge to guess the "right" way.	"The response provides a technically valid alternative to track.label."	"The response provides an alternative **such as using the videoKind property** to distinguish the stream."
Placeholder Language	Using "filler" phrases that sound professional but lack technical "teeth."	"The response follows industry best practices for error handling."	"The response includes a **try-except block** that catches the specific FileNotFoundError."
Circular Logic	Defining the rubric by the success of the outcome rather than the steps to get there.	"The code works correctly and produces the right output."	"The code uses a list comprehension to filter the data, ensuring the output list contains only integers."
High-Weight Cosmetics	Giving too much importance to formatting (Quality of Life) instead of technical logic.	"The response uses perfectly formatted headers and bold text."	"The response uses Markdown headers to separate the 'Setup' and 'Execution' phases for readability."
QoL rubrics passing All 4 models	The project attempter instructions explicitly say that "Quality-of-Life rubric evaluates the differences between the responses. **A good QoL criterion cannot be passed by all four responses."** QoL rubrics MUST evaluate the differences between responses.	a QoL rubric that all 4 responses pass or all 4 fail.	QoL rubrics MUST evaluate the differences between responses. Check: Ask yourself: "Does this rubric actually help distinguish which model is better?" 🔍

02/13 TLDR: Stop using adjectives or vague words, start using technical nouns and give concrete examples for specific methods. 🎯

Example: Don't say "appropriate tech stack"; say "uses SQLite and Python." ✅
Don't say "uses a valid method"; say "uses the videoKind property." ✅
Don't say "explains clearly"; say "identifies the LZ-String algorithm." ✅

But also watch out for being too prescrptive, you should give multiple examples if the model uses different ways to implement a code, don't limited the model to a single method!!

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-02-25T01:18:45.862Z